Classes and Objects using Swag Labs Login Page in TypeScript:

In a Swag Labs login page, we can use a class to represent the login page and an object to access its login functionality.

1. **Playwright Example:**

Define the class and create its object in the same file.

```
import { test, Page, Locator } from '@playwright/test';
class LoginPage {
constructor(private page: Page) {}
async login(user: string, pass: string)
{
await this.page.locator('#user-name').fill(user);
await this.page.locator('#password').fill(pass);
await this.page.locator('#login-button').click();
}  }

test('Swag Labs Login', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
const loginobj = new LoginPage(page);
await loginobj.login('standard_user', 'secret_sauce');
await loginobj.login('invalid_username', 'secret_sauce');
await loginobj.login('standard_user', invalid_pwd');
});
```

Define the class and create its object in different files.

```
project/
├── pages/
│   └── LoginPage.ts
└── tests/
    └── login.spec.ts
```

LoginPage.ts

```
import { Page, Locator } from '@playwright/test';
export default class LoginPage {
   username: Locator;
   password: Locator;
```

```
    loginButton: Locator;
    constructor(private page: Page) {
        this.username = page.locator('#user-name');
        this.password = page.locator('#password');
        this.loginButton = page.locator('#login-button');
    }
    async login(user: string, pass: string) {
        await this.username.fill(user);
        await this.password.fill(pass);
        await this.loginButton.click();
    }   }
```

login.spec.ts

```
import { test } from '@playwright/test';
import LoginPage from '../pages/LoginPage';
test('Swag Labs Login', async ({ page }) => {
    await page.goto('https://www.saucedemo.com/');
    const login = new LoginPage(page);
    await login.login('standard_user', 'secret_sauce');
    await login.login('standard_user', invalid_pwd');
});
```

Swag Labs Login Page using Classes, Objects, Constructors, and Methods in TypeScript with Playwright.

1. LoginPage.ts — Class with Methods

```
import { Page } from '@playwright/test';
export class LoginPage {
    page: Page;
    constructor(page: Page) {
        this.page = page;
    }
    async enterUsername(username: string) {  //Method-1
        await this.page.locator('#user-name').fill(username);
    }
    async enterPassword(password: string) { //Method-2
        await this.page.locator('#password').fill(password);
```

```
  }
  async clickLogin() { //Method-3
    await this.page.locator('#login-button').click();
  }    }
```

2. login.spec.ts — Create Object and Call Methods

```
import { test } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login', async ({ page }) => {
  await page.goto('https://www.saucedemo.com/');
  const login = new LoginPage(page);
  await login.enterUsername('standard_user');
  await login.enterPassword('secret_sauce');
  await login.clickLogin();
});
```

Inheritance in Playwright:

Inheritance allows one class (child class) to reuse the properties and methods of another class (parent class). In this example, we use the Swag Labs login page to demonstrate inheritance with Playwright.

BasePage.ts — Parent Class

```
import { Page } from '@playwright/test';
export class BasePage {
  page: Page;
  constructor(page: Page) {
    this.page = page;
  }
  async openWebsite() {
    await this.page.goto('https://www.saucedemo.com/');
  }   }
```

LoginPage.ts — Child Class

```
import { Page } from '@playwright/test';
import { BasePage } from './BasePage';
export class LoginPage extends BasePage {  //Child1 extends Parent
constructor(page: Page) {
super(page);
}
```

```
async enterUsername(username: string) {
await this.page.locator('#user-name').fill(username);
}
async enterPassword(password: string) {
await this.page.locator('#password').fill(password);
}
async clickLogin() {
await this.page.locator('#login-button').click();
}  }
```

login.spec.ts — Create Object and Call Methods

```
import { test } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login', async ({ page }) => {
   const login = new LoginPage(page);
   await login.openWebsite();
   await login.enterUsername('standard_user');
   await login.enterPassword('secret_sauce');
   await login.clickLogin();
});
```

Method Overloading in Playwright:

Method overloading means the same method name can accept different numbers or types of parameters.

1. LoginPage.ts

```
import { Page } from '@playwright/test';
export class LoginPage {
constructor(private page: Page) {}
async login(username: string): Promise<void>;
async login(username: string, password: string): Promise<void>;
async login(username: string, password?: string): Promise<void> {

await this.page.locator('#user-name').fill(username);
if (password !== undefined) {
await this.page.locator('#password').fill(password);
await this.page.locator('#login-button').click();
}  }  }
```

login.spec.ts

```
import { test } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Method Overloading', async ({ page }) => {
await page.goto('https://www.saucedemo.com/');
const login = new LoginPage(page);
await login.login('standard_user');
await login.login('standard_user', 'secret_sauce');
});
```

Method Overriding in Playwright:

Method overriding means the child class declares a method with the same name as the parent class and provides its own implementation.

1. Parent.ts:

```
import { Page } from '@playwright/test';
export class Parent {
   constructor(protected page: Page) {}
   async login() {
      await this.page.goto('https://www.saucedemo1.com/'); //Build1
      console.log('Parent login method');
   }  }
```

2. LoginPage.ts

```
import { Page } from '@playwright/test';
import { Parent } from './Parent';
export class LoginPage extends Parent {
   constructor(page: Page) {
      super(page);
   }
   override async login() {
      await this.page.goto('https://www.saucedemo.com/');
      await this.page.locator('#user-name').fill('standard_user');
      await this.page.locator('#password').fill('secret_sauce');
      await this.page.locator('#login-button').click();

      console.log('Swag Labs login successful');
```

```
  } }
```

3. login.spec.ts

```
import { test, expect } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login using Method Overriding', async ({ page }) => {
const login = new LoginPage(page);
await login.login();
await expect(page).toHaveURL(/inventory\.html/);
});
```

Data Abstraction in Playwright:

Abstraction means hiding implementation details and exposing only the essential functionality to the user. In TypeScript, we can demonstrate abstraction using an abstract class and an abstract method.

BasePage.ts — Abstract Parent Class

```
import { Page } from '@playwright/test';
export abstract class BasePage {
constructor(protected page: Page) {}

abstract login(): Promise<void>;
}
```

2. LoginPage.ts

```
import { Page } from '@playwright/test';
import { Parent } from './Parent';
export class LoginPage extends Parent {
constructor(page: Page) {
super(page);
}
override async login() {
await this.page.goto('https://www.saucedemo.com/');
await this.page.locator('#user-name').fill('standard_user');
await this.page.locator('#password').fill('secret_sauce');
await this.page.locator('#login-button').click();
console.log('Swag Labs login successful');
} }
```

3. login.spec.ts — Test Script

```
import { test, expect } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login using Abstraction', async ({ page }) => {
   const login = new LoginPage(page);
   await login.login();
   await expect(page).toHaveURL(/inventory\.html/);
});
```

Interface in Playwright:

An interface defines a contract that a class must follow. In this example, the interface declares the login method, and the LoginPage class implements it.

1. ILoginPage.ts — Interface

```
export interface ILoginPage {
login(): Promise<void>;  //Un-Implemented
}
```

2. LoginPage.ts — Implement Interface

```
import { Page } from '@playwright/test';
import { ILoginPage } from './ILoginPage';
export class LoginPage implements ILoginPage {
constructor(private page: Page) {}
async login(): Promise<void> {
await this.page.goto('https://www.saucedemo.com/');
await this.page.locator('#user-name').fill('standard_user');
await this.page.locator('#password').fill('secret_sauce');
await this.page.locator('#login-button').click();
}  }
```

3. login.spec.ts — Test Script

```
import { test, expect } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login using Interface', async ({ page }) => {
const login = new LoginPage(page);
await login.login();
await expect(page).toHaveURL(/inventory\.html/);
});
```

Encapsulation in Playwright:

Store the locators as private properties inside the LoginPage class and use them in public methods. This demonstrates encapsulation in Playwright.

1. LoginPage.ts

```
import { Page, Locator } from '@playwright/test';
export class LoginPage {
private username: Locator;
private password: Locator;
private loginButton: Locator;
constructor(private page: Page) {
this.username = page.locator('#user-name');
this.password = page.locator('#password');
this.loginButton = page.locator('#login-button');
}
async login(username: string, password: string) {
await this.page.goto('https://www.saucedemo.com/');
await this.username.fill(username);
await this.password.fill(password);
await this.loginButton.click();
} }
```

2. login.spec.ts

```
import { test, expect } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login', async ({ page }) => {
const login = new LoginPage(page);
await login.login('standard_user', 'secret_sauce');
await expect(page).toHaveURL(/inventory\.html/);
});
```

Exception Handling in Playwright:

In TypeScript, exception handling uses try, catch, and optionally finally to handle errors without abruptly stopping the execution flow.

LoginPage.ts

```
import { Page } from '@playwright/test';
export class LoginPage {
constructor(public page: Page) {}
async login(username: string, password: string) {
try {
await this.page.goto('https://www.saucedemo.com/');
await this.page.locator('#user-name').fill(username);
```

```
await this.page.locator('#password').fill(password);
await this.page.locator('#login-button').click();
console.log('Login successful');
}
catch (error) {
console.log('Login failed:', error);
throw error;
}
finally {
console.log('Login execution completed');
}       }    }
```

2. login.spec.ts

```
import { test, expect } from '@playwright/test';
import { LoginPage } from './LoginPage';
test('Swag Labs Login', async ({ page }) => {
const login = new LoginPage(page);
await login.login('standard_user', 'secret_sauce');
await expect(page).toHaveURL(/inventory\.html/);
});
```

****